



西南大学 2023 年免费的真题精讲课程获取
详情请加入西南大学考研 QQ 交流群：567961850 获取！
新硕学长微信：swu1713

2023 年西南大学招收硕士研究生入学考试试题

(考生注意：全部答案必须写在答题纸上否则后果自负！)

考试科目代码：808

考试科目：计算机专业基础综合

数据结构

一、判断题 (每题 1 分, 共 10 分)

- 1、树、栈、队列是数据结构中的储存结构
- 2、时间复杂度为 $O(n \log_2 n)$ 的算法不一定比 $O(n^2)$ 运行时间短
- 3、二叉树就是度为 2 的树
- 4、在速度慢的输入输出设备与速度快的处理器之间要用队列辅助
- 5、一颗树的跟结点大于其左孩子结点, 小于其右孩子结点, 则其是一颗二叉树
- 6、KMP 算法是先对主串计算 next 数组, 然后再用模式串匹配主串

二、填空题 (每题 1 分, 共 10 分)

- 1、假设二叉树中度为 0 的结点个数为 n_0 , 度为 1 的结点个数为 n_1 , 度为 2 的结点个数为 _____, 一个含有 n 个结点的二叉树空链域为 _____
- 2、常见排序算法 (快速、选择、归并、冒泡) 中, _____ 算法的空间复杂度最高, 其空间复杂度为 _____
- 3、快速排序算法在 _____ 情况下性能最差, 时间复杂度为 _____
- 4、图的各边权值相同, 不用 Dijkstra 算法, 求一个顶点到其他顶点的最短路径还可以使用 () 方法, 需要使用 () 作为辅助空间
- 5、若无向图中所有边权为 1, 则计算单源最短路径可不用迪杰斯特拉算法, 而采用 (), 这样做需要使用 () 作为辅助空间

三、简答题 (15 分)

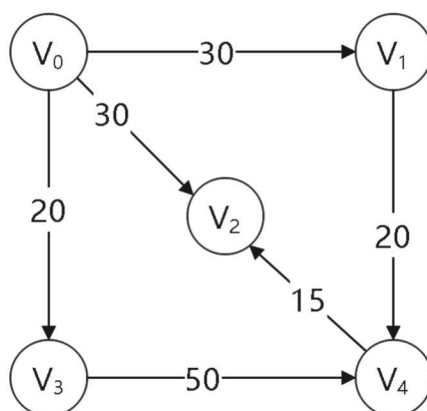
- 1、单链表引入头结点有什么好处? 列举两个具体操作 (6)
- 2、构造一个哈希表的基本要素有什么? 平均查找长度和什么因素有关? 当从哈希表中删



除元素时，需要移动大量元素，为了提高删除效率，可以采用什么删除策略 (9)

四、大题 (10 分)

使用 Dijkstra 算法计算 V_0 到其他结点的最短距离。



五、(15 分)

高斯消元实现多元线性方程组的化简分为两步，第一步向下消元获得行阶梯形矩阵，第二步向上消元获得行最简矩阵，求解三元一次方程组，先向下消元再向上消元

- 1、写出伪代码，指出主要时间消耗在哪？
- 2、求向下消元的时间复杂度
- 3、求高斯消元总的时间复杂度

六、(15 分)

在斐波那契数列中，除前两项外，每一项都满足 $F_n = F_{n-1} + F_{n-2}$

$$F_n = F_{n-1} + F_{n-2}$$

- 1、递归实现斐波那契数列，并分析时间复杂度。
- 2、运用矩阵可以实现计算斐波那契数列，使用C/C++语言编写计算矩阵幂的算法，常用数据结构可以直接使用，但需要说明，实现算法的执行过程。

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$$

$$A^2 = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} F_3 & F_2 \\ F_2 & F_1 \end{pmatrix}$$

$$A^n = \begin{pmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{pmatrix}$$

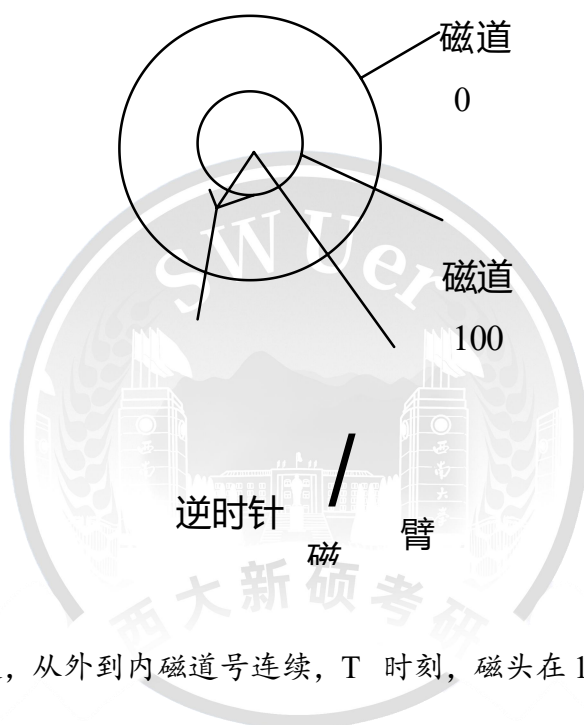


操作系统

七、

- 1、介绍进程的三种基本状态及状态转换，并说明状态转换的出发事件
- 2、PCB 中除了存储进程状态这一重要信息，还存储哪些信息，至少列举两种

八、磁盘调度



最外侧0 号磁道，从外到内磁道号连续，T 时刻，磁头在 100 号磁道，磁头逆时针旋转

- 1、磁盘转速7200r/min，求平均延迟时间
- 2、有一个磁道访问请求为 120、80、40、60，求在改进型 SCAN 算法下的磁盘调度序列，并计算平均寻道长度。

九、一个文件的索引表中有4 个直接地址索引，2 个一级地址索引，1 个二级地址索引，地址项占4B，索引块和磁盘块大小都是256KB，求文件的最大长度



十、

- 1、说明LRU 算法的思想
- 2、用栈怎么实现LRU 算法
- 3、访问一个虚拟地址，为什么会发生缺页

十一、

进程按优先级调度，数字越大优先级越高

进程号	等待时间(ms)	剩余运行时间(ms)	优先级
P1	15	10	10
P2	40	20	30
P3	25	30	20

- 1、什么是进程周转时间，什么是系统平均周转时间
- 2、不计进程调度时间开销，计算平均周转时间
- 3、进程调度时间 1ms，计算平均周转时间

十二、页面大小为 4 KB，一次内存的访问时间是 100 ns，一次快表（TLB）的访问时间是 10 ns，处理一次缺页的平均时间为 108 ns（已含更新 TLB 和页表的时间），进程的驻留集大小固定为 2，采用最近最少使用置换算法（LRU）和局部淘汰策略。假设 TLB 初始为空，地址转换时先访问 TLB，若 TLB 未命中，再访问页表（忽略访问页表之后的 TLB 更新时间）。有效位为 0 表示页面不在内存，产生缺页中断，缺页中断处理后，返回到产生缺页中断的指令处重新执行。设有虚地址访问序列 2362H、1565H、25A5H，请问：

页号	页框号	有效位（存在位）
0	101H	1
1	—	0
2	254H	1

- (1) 依次访问上述三个虚地址，各需多少时间？给出计算过程。
- (2) 基于上述访问序列，虚地址 1565H 的物理地址是多少？请说明理由。



2023 年西南大学招收硕士研究生入学考试试题参考答案

(考生注意：全部答案必须写在答题纸上否则后果自负！)

考试科目代码：808

考试科目：计算机专业基础综合

数据结构

一、判断题 (每题 1 分, 共 10 分)

1、错; 2、对; 3、错; 4、对; 5、错; 6、错

二、填空题 (每题 1 分, 共 10 分)

1、 $m-1, n+1$

2、归并, $O(n)$

3、正序或逆序, $O(n^2)$

4、floyd 算法, 矩阵

5、BFS, 队列

三、简答题 (15 分)

1、使得在链表头部的操作(如:插入删除等)与在链表中部与尾部一致;使非空链表与空链表的操作统一。

①使用头结点,在表首插入删除操作的都是 $head \rightarrow next$,和其他位置的操作统一。

②获取表首元素可以直接 $return head \rightarrow next$,如果没有头结点,需要判断是 NULL 还是具体的元素。

2、要素: 哈希函数, 冲突处理方法。

ASL 有关: 哈希表中的元素数量, 哈希函数的质量, 哈希表的大小, 冲突解决方法
删除策略: 使用拉链法表示。

四、大题 (10 分)

顶点	初始化	第一轮	第二轮	第三轮	第四轮
1	$30(v_0 \rightarrow v_1)$	$30(v_0 \rightarrow v_1)$	$30(v_0 \rightarrow v_1)$	$30(v_0 \rightarrow v_1)$	$30(v_0 \rightarrow v_1)$
2	$30(v_0 \rightarrow v_2)$	$30(v_0 \rightarrow v_2)$	$30(v_0 \rightarrow v_2)$	$30(v_0 \rightarrow v_2)$	$30(v_0 \rightarrow v_2)$



3	$20(v_0 \rightarrow v_3)$	$20(v_0 \rightarrow v_3)$	$20(v_0 \rightarrow v_3)$	$20(v_0 \rightarrow v_3)$	$20(v_0 \rightarrow v_3)$
4	∞	$70(v_0 \rightarrow v_3 \rightarrow v_4)$	$50(v_0 \rightarrow v_1 \rightarrow v_4)$	$50(v_0 \rightarrow v_1 \rightarrow v_4)$	$50(v_0 \rightarrow v_1 \rightarrow v_4)$
集合 S	$\{0\}$	$\{0, 3\}$	$\{0, 3, 1\}$	$\{0, 3, 1, 2\}$	$\{0, 3, 1, 2, 4\}$

五、(15 分)

1.matrix Gaussian(matrix A) {

```

    for(int i = 0; i < A.size; i++) {        // 向下消元
        for(int j = i; j < A.size; j++) {
            A.data[i][j] /= A.data[i][i];    // 本行首元素置为 1
        }
        for(int j = i; j < A.size; j++) {
            for(int k = i; k < A.size; k++) {
                // 其他行第 i 列置为 0
                A.data[j][k] -= A.data[j][i] * A.data[i][k];
            }
        }
    }

    for(int i = size - 1; i >= 0; i--) {    // 向上消元
        for(int j = i; j >= 0; j--) {
            A.data[j][i] -= A.data[i][i];
        }
    }
}

```

主要时间消耗在向下消元。

2. $O(n^3)$

3.由于向上消元部分时间复杂度只有 $O(n^2)$ ，所以总时间复杂度为 $O(n^3)$ 。

六、

1.



```
int Fib(int n) {
    if(n == 0) return 0;
    if(n == 1) return 1;

    return Fib(n - 1) + Fib(n -
2); }
```

时间复杂度为 $O(2^n)$ 。

2.

```
typedef struct
    matrix{
        int
        data[2][2];
        int
size; }matrix
x;
matrix mul(matrix A, matrix B) { // 矩
    阵乘法 matrix R;
    for(int i = 0; i < A.size; i++) {
        for(int j = 0; j < A.size; j++) {
            R.data[i][j] = 0;
            for(int k = 0; k < A.size; k++) {
                R.data[i][j] += A.data[i][k] *
                B.data[k][j]; }
        }
    }
    return R;
}
```

```
matrix Fib(int n)
{ matrix An;
```



```
if(n == 1) {  
    An.data = {1, 1, 1, 0};  
  
    return  
An; }  
  
// 用快速幂计  
算 An = Fib(n / 2);  
An = mul(An, An);  
if(n % 2 == 1) An = mul(An, Fib(1));  
  
return  
An; }
```

时间复杂度为 $O(\log_2 n)$ 。

操作系统

七、1.1) 就绪(Ready)状态: 进程已分配到除 CPU 以外的所有必要的资源, 只要获得处理机便可立即执行, 这时的进程状态称为就绪状态。

2) 执行(Running)状态: 当一个进程已获得必要的资源, 并占有中央处理机执行时进程所处的状态。

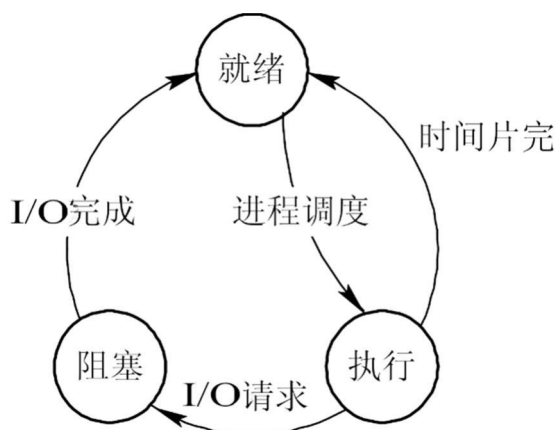
3) 阻塞(Blocked)状态: 正在执行的进程, 由于发生某事件(如等待输入/输出完成)而暂时无法执行下去时, 便放弃处理机, 此时进程所处的状态。

就绪→执行: 处于就绪状态的进程, 当进程调度程序为之分配了处理机后, 该进程便由就绪状态转变成执行状态。

执行→就绪: 处于执行状态的进程在其执行过程中, 因分配给它的一个时间片已用完而不得不让出处理机, 于是进程从执行状态转变成就绪状态。

执行→阻塞: 正在执行的进程因等待某种事件发生而无法继续执行时, 便从执行状态变成阻塞状态。

阻塞→就绪: 处于阻塞状态的进程, 若其等待的事件已经发生, 于是进程由阻塞状态转变为就绪状态。



2、1.进程id。系统中每个进程有唯一的id，在c语言中用pid_t类型表示，其实就是一个非负整数。

2. (进程的状态，有就绪，运行，挂起，停止等状态)

3.进程切换时需要保存和恢复的一些CPU寄存器

4.描述虚拟地址空间的信息。

5.描述控制终端的信息。

6.当前工作目录 (Current Working Directory)

7.umask 掩码

8.文件描述符表，包含很多指向结构体的指针

9.和信号相关的信息

10.用户id和组id。

11.会话 (Session) 和进程组

12.进程可以使用的资源上限 (Resource Limit)

八、1) $60 / 7200 * 0.5 = 0.00417 = 4.17 \text{ ms}$

2) 磁道访问序列为 80, 60, 40, 120，平均寻道时间为 55。

九、每个索引块可以存放的盘块个数： $256\text{KB}/4\text{B}=64\text{K}$

故文件的最大长度是 $(4+2*64\text{K}+64\text{K}*64\text{K})*256\text{KB} = 1\text{PB}+32\text{GB}+1\text{MB}$

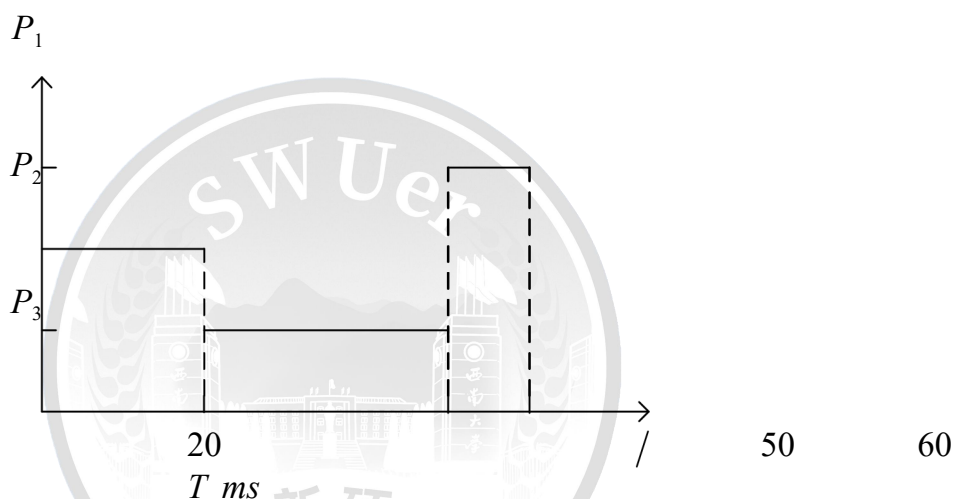
十、1、LRU 是Least Recently Used 的缩写，即最近最少使用页面置换算法，是为虚拟页式存储管理服务的，是根据页面调入内存后的使用情况进行决策了。由于无法预测各页面将来的使用情况，只能利用“最近的过去”作为“最近的将来”的近似，因此，LRU算法就是将最近最久未使用的页面予以淘汰。



2、可以用一个特殊的栈来保存当前正在使用的各个页面的页面号。当一个新的进程访问某页面时，便将该页面号压入栈顶，其他的页面号往栈底移，如果内存不够，则将栈底的页面号移除。这样，栈顶始终是最新被访问的页面的编号，而栈底则是最近最久未访问的页面的页面号。

3、在操作系统中，程序执行时需要访问的内存可能未加载到物理内存中，需要从磁盘读入，此时就会发生缺页中断。

十一、运行顺序如下：



1、进程周转时间=进程完成时刻-进程到达时刻

系统平均周转时间：进程总周转事件/进程个数

2、调度顺序为P2、P3、P1，平均周转时间 $= (40+20+25+50+15+60)/3=70\text{ms}$

3、调度顺序为P2、P3、P1，平均周转时间 $= (40+21+25+52+15+63)/3=72\text{ms}$

十二、1、根据页式管理的工作原理，应先考虑页面大小，以便将页号和页内位移分解出来。页面大小为 $4\text{KB}=2^{12}\text{B}$ ，则得到页内位移占虚地址的低 12 位，页号占剩余高位。可得三个虚地址的页号 P 如下(十六进制的一位数字转换成 4 位二进制，因此，十六进制的低 12 位正好为页内位移，最高位为页号)。

2362H: 页号 $P=2$ ，有效位为 1，存在内存中。先访问快表 10ns，因初始为空，不在快表中，因此，需要访问页表 100ns 得到页框号，合成物理地址后访问主存 100ns，共计 $10\text{ns} + 100\text{ns} + 100\text{ns} = 210\text{ns}$ 。



1565H: 页号 $P=1$, 有效位为 0, 不存在内存中。先访问快表 10ns, 落空, 访问页表 100ns 落空, 进行缺页中断处理 108ns, 合成物理地址后访问主存 100ns, 共计 $10ns + 100ns + 108ns + 100ns = 318ns$ 。

25A5H: 页号 $P=2$, 有效位为 1, 存在内存中。访问快表, 因第一次访问已将该页号放入快表, 因此花费 10ns 便可合成物理地址, 访问主存 100ns, 共计 $10ns + 100ns = 110ns$ 。

2、当访问虚地址 1565H 时, 产生缺页中断, 合法驻留集为 2, 必须从页表中淘汰一个页面, 根据题目的最近最少使用置换算法, 应淘汰 0 号页面, 因此 1565H 的对应页框号为 101H。由此可得 1565H 的物理地址为 101565H。





西南大学 2024 年免费的真题精讲课程获取
详情请加入西南大学考研 QQ 交流群: 567961850 获取!
新硕学长微信: swu1713

2024 年西南大学招收硕士研究生入学考试试题

(考生注意: 全部答案必须写在答题纸上否则后果自负!)

数据结构

一、判断题(每题 1 分, 共 10 分)

- 1、设一棵树 T 可以转换成二叉树 BT , 则在二叉树 BT 中, 根结点一定无右子树。
- 2、不论线性表采用顺序存储结构还是链式存储结构, 删除值为 X 的结点的平均时间复杂度均为 $O(n)$ 。
- 3、层序遍历小根堆可以得到一个有序序列。
- 4、冒泡排序在初始关键字序列逆序情况下, 交换次数最多。
- 5、二叉树就是度为 2 的树。
- 6、若一个非叶结点是某二叉树的中序遍历的最后一个结点, 则他必是该二叉树先序遍历的最后一个结点。
- 7、如果一颗二叉树每个结点的关键字值大于左孩子, 又小于右孩子, 则是二叉排序树。
- 8、二分查找时, 查找失败的 $ASL \approx \log_2(n)+1$ 。
- 9、与 Prim 算法相比, Kruskal 算法适合稀疏图。
- 10、选择排序的时间复杂度与待排序序列无关。

二、填空题(每题 1 分, 共 10 分)

- 1、折半查找的要求是_____, 具有_____和_____的特点。适合条件



为和_____。

2、哈希函数分布均匀时，_____和_____影响平均查找长度。

3、路径权值相等，除了用迪杰斯特拉算法求最短路径，还可以使用_____算法来求最短路径，存储结构是_____。

4、在二叉排序树中要删除根结点，要求用_____或_____来代替根结点。

5、有先序中序后序，要构造出二叉树可以用_____组合或_____组合。

三、一数据加密算法的密钥为 56 位二进制数据，在 1997 年时，破解该密钥需要花费 140 天，GROVE 算法将时间复杂度降低到了 $O(\sqrt{n})$ 。用 1997 年的算力和时间，GROVE 算法可以破解多少位密钥？GROVE 破解 56 位密钥，需要多长时间？（10 分）

四、比较、分析图的存储结构（邻接表法、邻接矩阵法），假设顶点为 n 个，边有 e 条，复杂度用 O 表示法，分析如下图表：（15 分）、

	邻接表	邻接矩阵
存储结构		
唯一性		
空间复杂度		
访问某一顶点时间复杂度		
访问某条边时间复杂度		
访问一个顶点所有边时间复杂度		
遍历图时间复杂度		

五、矩阵乘方定义 $B=A*A$ ，可以使用累计相乘法计算， A^3 可以转化为 $A*A*A=B*A$ 。

(1) 累计相乘法代码如下所示，求出 A^n 的时间复杂度（用 O 表示）。

```
for(int i = 0; i < n; ++i) {
    for(int j = 0; j < n; ++j) {
        for(int k = 0; k < n; ++k) {
            B[i][j] = B[i][j] + A[i][k] * A[k][j];
        }
    }
}
```



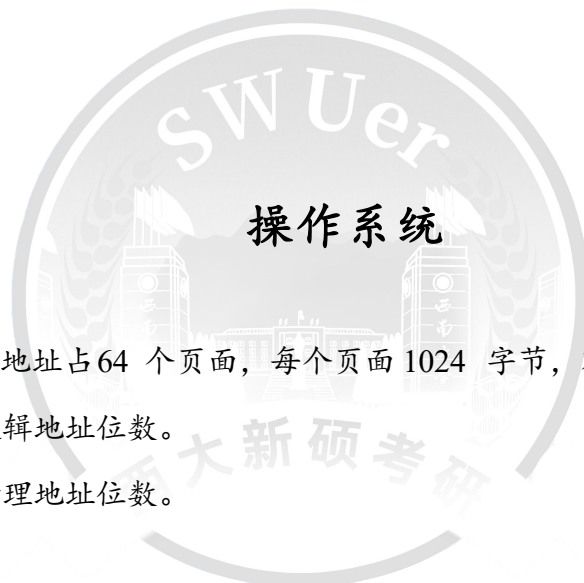
}

}

}

(2) 设计更高效率的算法计算矩阵乘方 A^n , 指出数据结构, 描述思路, 具体实现代码, 及分析时间复杂度。

六、使用Kruskal 算法时, 如何判断有无回路。



七、某操作系统逻辑地址占64 个页面, 每个页面 1024 字节, 对应有32 个物理页框。

(1) 求出最小逻辑地址位数。

(2) 求出最小物理地址位数。

八、有一台CPU (单核) 和一台输入机、并发环境中有P1、P2、P3 按先来后到顺序并发执行, 执行同样的任务T, 包括信息录入T1, 计算T2 和输出T3, 每个时间分别是2ms、3ms、4ms, 问执行完三个进程共需要多少时间, CPU 利用率是多少。(10 分)

九、简述死锁必要条件, 为什么从小到大编码可以避免死锁。(10 分)

十、有一个多级目录系统:

--SOC1

--SOC2

--Dec2



--Dec3

--Dec1

文件名	文件索引结点对应的物理块
SOC1	2
SOC2	4
Dec1	7
Dec2	8
Dec3	9

(1) 写出SOC1 目录下的目录表。

(2) 若采用混合索引, 有 10 个直接索引, 1 个一级间接索引, 1 个二级间接索引, 一个物理块为5KB, 一个地址占4B。问有一个5MB 的文件, 用直接文件和一级间接索引能否存储下? 为什么?

十一、系统采用分页、页大小200B, 一个整型占1B, 数组A[101][99]按行存储, 最多给数组A 分配两个物理页框, 采用最近最久未使用LRU 算法。初始时给A 分配了前2个页面。

```
for(int i = 0; i < 101; ++i)
```

```
for(int j = 0; j < 99; ++j)
```

```
A[i][j] = 0
```

(1) 初始化A 缺页多少次(发生了多少次换页), 并说明理由。

(2) 初始化完成后, 有多少A 的元素最终常驻内存?

十二、

(1) 一个并发运行的环境, 对一个临界资源互斥访问, 代码如下:

```
int decrease() {
```

```
    if(available < 1) {
```

```
        return -1;
```

```
    } else {
```

```
        available -= 1;
```



```
return 0;

}

}

int increase() {

    available += 1;

    return 0;

}
```

其中开锁函数release(lock)和上锁函数lock()已定义好，请修改上述两个函数，解决他们之间的竞争关系。

(2) 系统使用 swap(a, 2)源于解决互斥、同步关系，请问_____处完善“FALSE”或“TRUE”，并说明理由。

代码：

```
int lock = FLASE;

...

// 进入区

int key = TRUE;

while(key

    == _____)

    { swap(key, lock);

    }

...

// 访问

int      lock

= _____;

// 退出区

...
```



2024 年西南大学招收硕士研究生入学考试试题参考答案

(考生注意: 全部答案必须写在答题纸上否则后果自负!)

数据结构

一、判断题(每题 1 分, 共 10 分)

- 1、对。在孩子兄弟表示法的二叉树中, 只有森林才会出现根结点有右子树的情况。
- 2、对。
- 3、×。小根堆只满足任一根结点值小于其所有子孙节点。
- 4、对。
- 5、错。二叉树是度小于等于 2 的有序树。
- 6、错。如果中序遍历的最后一个是非叶结点, 只能说明其无右子树。
- 7、错。必须递归满足根节点大于等于所有左子树结点, 小于等于所有右子树结点。
- 8、对。
- 9、对。
- 10、对。选择排序无论序列初始状态如何均需完整遍历序列。

二、填空题(每题 1 分, 共 10 分)

- 1、折半查找的要求是 有序顺序表, 具有 对数时间复杂度 和 常数空间复杂度 的特点。适合条件为 不常修改 和 查找频繁。
- 2、哈希函数分布均匀时, 冲突解决策略 和 装填因子 影响平均查找长度。
- 3、路径权值相等, 除了用迪杰斯特拉算法求最短路径, 还可以使用 Floyd (或 BFS) 算法来求最短路径, 存储结构是 矩阵 (或队列)。
- 4、在二叉排序树中要删除根结点, 要求用 左子树的最大结点 或 右子树的最小结点 来代替根结点。
- 5、有先序中序后序, 要构造出二叉树可以用 先序+中序 组合或 中序+后序 组合。

三、

设算力为 P, 由题意得 $2^{56}/P=140$ 天

- 1、设采用 GROVE 算法, 用 1997 年的算力可以破解 m 位密钥
则 $\sqrt{2^m}/P=140$ 天, 解得 $m=112$ 位

- 2、破解 56 位密钥时, GROVE 算法需要 $\sqrt{2^{56}}/P=0.045$ 秒

【b 站/公众号/小红书搜一搜: 西南大学新硕学长】



四、

	邻接表	邻接矩阵
存储结构	顺序表+链表	顺序表
唯一性	不唯一	唯一
空间复杂度	$O(n+e)$	$O(n^2)$
访问某一顶点时间复杂度	入度为 $O(n+e)$ ，出度为 $O(e)$	$O(n)$
访问某条边时间复杂度	遍历该边起点的边表， $O(e)$	$O(1)$
访问一个顶点所有边时间复杂度	入度为 $O(n+e)$ ，出度为 $O(e)$	$O(n)$
遍历图时间复杂度	$O(n+e)$	$O(n^2)$

五、(1) 每一次矩阵乘法的时间复杂度为 $O(n^3)$ ，由于 A^n 需要进行 $n-1$ 次矩阵相乘，故总时间复杂度为 $O(n^4)$ 。

(2) 可以用快速幂的思想来实现矩阵乘法，除了矩阵无需额外的数据结构，设矩阵为 k 阶，则矩阵快速幂的总时间复杂度为 $O(n^3 \cdot \log_2 k)$ 。

如下是基于递归实现的矩阵快速幂算法：

```
typedef struct mat {
    int data[MAX][MAX]; // 矩阵数据
    int k; // 矩阵阶
};

mat mul(mat A, mat B) { // 矩阵乘法
    mat R;
    for(int i = 0; i < A.k; i++) {
        for(int j = 0; j < A.k; j++) {
            R.data[i][j] = 0;
            for(int k = 0; k < A.size; k++) {
                R.data[i][j] += A.data[i][k] * B.data[k][j];
            }
        }
    }
}
```



```
return R;

}

mat pow(mat A, int n) { // 计算矩阵的n 次方
    if(n == 0) return E(A.k); // 返回单位矩阵
    if(n == 1) return A; // 返回A
    mat A2 = mul(A, A); // 计算A2
    if(n % 2 == 0) return pow(A2, n / 2); // 偶数次方
    else return mul(A, pow(A2, n/2)); // 奇数次方, 补一个A 相
    乘 }
}
```

六、用并查集判断边的两个顶点是否来自于同一个连通分量,如果是就说明有回路。int

```
set[maxSize] = {-1}; // 并查集结构, 记录顶点所在的连通分量集合
int findSet(int k) {
    if(set[k] == -1) set[k] = k; //初始化所在连通分量为自己
    set[k] = findSet(set[k]); // 递归查找所在编号并更新
    return set[k];
}

bool hasCircle(int i, int j){ // 判断是否有回路
    int setI = findSet(i), setJ = findSet(j); //查找所在连通分
    量 set[i] = setI;
    set[j] = setJ;
    if(setI == setJ) return true; // 有回路
    else {
        set[setJ] = setI; // 没有就放到同一个连通
        分量
        return false;
    }
}
}
```



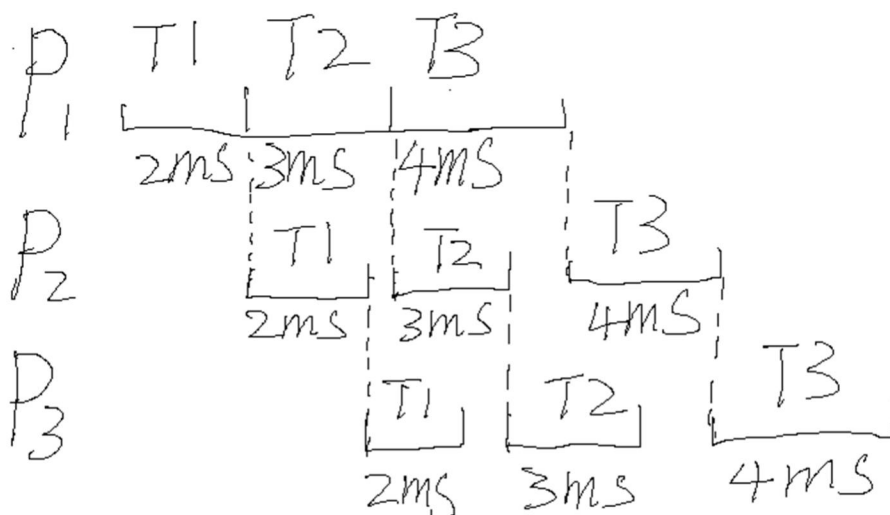
操作系统

七、每个页面 $1024=2^{10}$ 个字节，需要 10 位存储页内偏移。

(1) 系统有 $64=2^6$ 个页面，共需要 16 位逻辑地址。

(2) 系统有 $32=2^5$ 个页面，共需要 15 位逻辑地址。

八、程序执行时序如下图所示，总执行时间为 17ms。CPU 利用率为 $9\text{ms}/17\text{ms}=9/17$ 。



九、**死锁**：是指多个进程在运行过程中因争夺资源而造成的一种僵局，当进程处于这种僵持状态时，若无外力作用，它们都将无法再向前推进。

产生死锁的必要条件：

- ①**互斥**：指进程对所分配到的资源进行排它性使用，即在一段时间内某资源只由一个进程占用。
- ②**请求和保持**：指进程已经保持了至少一个资源，但又提出了新的资源请求，而该资源又已被其它进程占有，此时请求进程阻塞，但又对自己已获得的其它资源保持不放。
- ③**不剥夺**：指进程已获得的资源，在未使用完之前，不能被剥夺，只能在使用完时由自己释放。
- ④**循环等待**：指在发生死锁时，必然存在一个进程——资源的环形链。

从小到大编码，即采用资源有序分配，其基本思想是将系统中的所有资源顺序编号。破坏了死锁的循环等待条件。将紧缺的，稀少的采用较大的编号，在申请资源时必须按照编号的顺序进行（从小往大申请），一个进程只有获得较小编号的进程才能申请较大编号。



号的进程。

十、(1)

文件名	文件索引结点对应的物理块
SOC2	4
Dec3	9

(2) 一级间接索引可以指向 $5KB/4B=1280$ 个物理块，加上 10 个直接索引，共 1290 个物理块，对应 $1290*5KB=6450KB>5120KB(5MB)$ ，所以直接索引和一级间接索引可以存储下。

十一、数组共 $101*99=9999$ 个元素，共占用 $9999*1B=9999B$ 内存， $9999B/200B\approx 50$ 个物理块。

(1) 初始化时，由于已经分配了前 2 个物理块，所以缺页发生在后 48 个物理块，缺页 48 次。

(2) 最后 2 个物理块的元素会常驻内存，但是最后一个物理块只有 199 个元素，所以共 $200+199=399$ 个元素常驻内存。

十二、

(1)

```
int decrease() {  
    lock();  
    if(available < 1) {  
        release(lock);  
        return -1;  
    } else {  
        available -= 1;  
        release(lock);  
        return 0;  
    }  
}
```



}

```
int increase() {  
  
    lock();  
  
    available += 1;  
  
    release(lock);  
  
    return 0;  
  
}
```

(2) $key = \text{TRUE}$ 的时候才能进入循环, 表示可以上锁, swap 会对 $lock = \text{TRUE}$ 上锁, 执行结束后, 需要让 $lock = \text{FALSE}$, 将锁释放。

```
int lock = FALSE;  
  
...  
// 进入区  
int key = TRUE;  
while(key == TRUE) {  
  
    swap(key,  
lock); }  
...  
// 访问  
int lock = FALSE;  
  
// 退出区  
  
...
```



西南大学 2025 年免费的真题精讲课程获取
详情请加入西南大学考研 QQ 交流群: 567961850 获取!
新硕学长微信: swu1713

2025 年西南大学招收硕士研究生入学考试试题

(考生注意: 全部答案必须写在答题纸上否则后果自负!)

数据结构

一、填空题 (每题 2 分, 共 20 分)

- 1、有数组 $A[8][10]$, 起始地址是 100, 求 $A[4][5]$ 。
- 2、循环队列 $Q[20]$, 队头下标是 17, 队列长度 13, 求队尾下标。
- 3、 n 个结点的完全二叉树, 最少有多少个叶子结点。
- 4、二叉树给了中序和后序序列, 求其先序 (很小的一颗树)。
- 5、一个邻接矩阵, 求第 i 个结点的入度。
- 6、一颗 m 叉树, 度为 0 的结点数为 N_0 , 度为 1 的节点数为 N_1 ……度为 m 的结点数为 N_m , 求 N_0 是多少。
- 7、链式存储结构的二叉树, 问空链域数是多少, 其先序线索化后, 空链域是多少。
- 8、给了一串数组, 构造一个哈夫曼树, 计算 WPL。
- 9、计算时间复杂度 (似乎是下面这样)
 $i=1, j=0;$
 $\text{while}(i \leq n) \{$
 $i+=j;$
 $j++;$
 $\}$
- 10、一颗二叉树高为 h , 且没有度为 1 的结点, 问最少有多少个结点。

二、 n 个城市, 要想修路用最少的代价让每个城市联通, 请问用什么策略, 简述原理



【(数字记不清了)知道 A 城市和 B 城市的距离 391, A 到 C 是 496, B 到 C 是 179。已知 A 和 B 已经有高速公路, 现在要实现城市 A 和城市 B 和城市 C 的连通的最小距离是 179, 请问用什么算法实现】 (10 分)

三、dijkstra 能不能计算带有负权的图, 如果能详细说明, 如果不能, 请举例说明。(10 分)

四、用 S1、S2 两个栈实现队列的入队和出队操作(10 分)

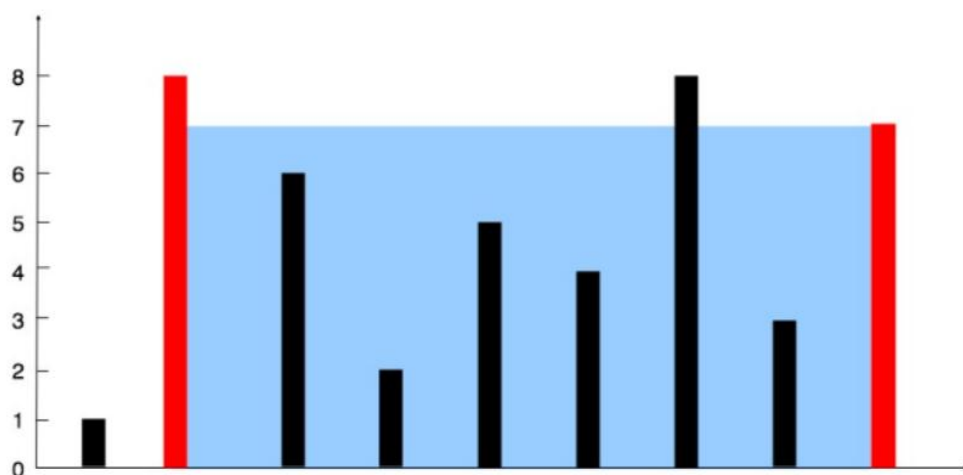
五、一个数组 nums, 给定一个 target 值, target 值能被该数组的俩个元素唯一表示【 $\text{nums}[i] + \text{nums}[j] = \text{target}$ 】, 不会有多个能表示 target 的一对数组, 请设计一道算法返回俩个相加等于 target 的值的元素的下标 (10 分)

(1) 给出算法思想

(2) 需要用的变量、数据结构说明, 时间复杂度尽可能低。

(3) 步骤和时间复杂度分析

六、给定一个长度为 n 的整数数组 height, 有 n 条垂线, 第 i 条线的两个端点是 (i, 0) 和 (i, height[i]), 找出其中的两条线, 使得他们与 x 轴共同构成的容器可以容纳最多的水, 返回容器可以存储的最大水量。(10 分)



输入: [1,8,6,2,5,4,8,3,7]

输出: 49

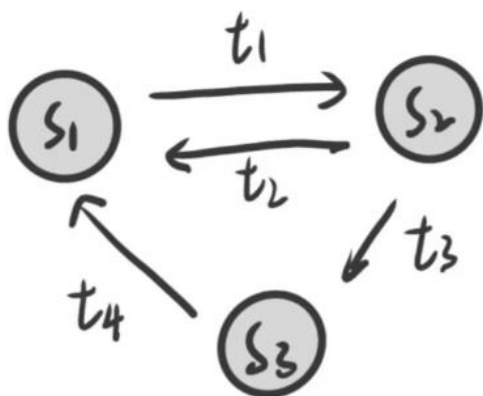
解释: 图中垂直线代表输入数组 [1,8,6,2,5,4,8,3,7]。在此情况下, 容器能够容纳水 (表示为蓝色部分) 的最大值为 49。



操作系统

七、索引结构存储文件，每个盘块大小为 4kb，32 个比特位作为盘块号。节点用 13 个条项做索引。10 个直接地址，后三个对应 1 级，2 级，3 级间接地址，请问最大的文件大小是多少？请写出计算过程。（10 分）

八、（10 分）



(1) S1, S2, S3 分别代表进程的什么状态？

(2) t1, t2, t3, t4 的触发条件是什么？

九、

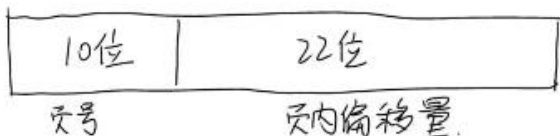
(1) 以下哪些内容可以由多个进程共享？哪些是进程单独占有？请做出选择并说明理由。（10 分，部分答对不给分）

- a. 寄存器值
- b. 堆内存（堆内容）
- c. 全局变量
- d. 栈内存

(2) 一个进程内的线程可共享上述哪些内容，不可共享上述哪些内容？并说明理由。

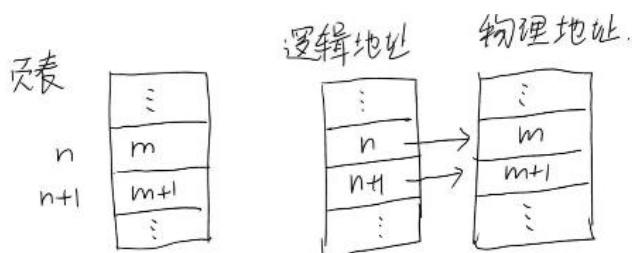
（10 分，部分答对不给分）

十、按字节编址，逻辑地址和物理地址都是 32 位，给出了一个地址结构，22 位页内偏移量，10 位页号（10 分）



(1) 一页多大? 一个页表项 4b, 存储所有的页需要多大连续空间? (结果用 KB 或 MB 表示)

(2)



两个逻辑上连续的页面 $n, n+1$ 存储在连续的两个物理块 m 和 $m+1$ 中, 给出了页框 m 的物理地址 $a1400000h$, 计算 $m+1$ 页框的物理初始地址, 分别计算 n 和 $n+1$ 中翻译地址的数据, 答案以 16 进制表出。

十一、(15 分)

	执行时间	截止时间	截止时间 - 执行时间 → 中断到地址和程序之间最大间隔
A	20ms	10ms	20ms
B	80	10ms	80
C	25	5ms	25

- (1) 在某种调度算法下, A、B、C 是否能在截止时间内完成【问空闲时间占的比率】。
- (2) 以 $C > B > A$ 的优先级, A 的最坏时间。
- (3) 以 $C > B > A$ 的优先级, A、B、C 是否能在截止时间内完成。

十二、[PV 操作] 4 个进程 R1、R2、W1、W2, 一个缓冲区 B, 用 PV 实现以下功能: (15 分)

- (1) R1 是用键盘输入一个数据至缓冲区 B, W1 负责打印输出 R1 输入的数据。
- (2) R2 是用磁盘读入一个数据至缓冲区 B, W2 负责打印输出 R2 输入的数据。
- (3) 当缓冲区 B 空时, $w1$ 和 $w2$ 都不能读数据。
- (4) 当缓冲区 B 存在数据时, R1 和 R2 便不能输入数据。



2025 年西南大学招收硕士研究生入学考试试题参考答案

(考生注意：全部答案必须写在答题纸上否则后果自负！)

数据结构

一、填空题。(20 分)

1、 $100+2*(4*10+5)=190$

2、 $(17+13)\%20=10$

3、 $n_2=n_0-1$, $n_0+n_1+n_2=n$, 即 $n_0=(n-n_1)/2$, 最少为 $(n-1)/2$ 个叶子节点

4、略

5、看第 i 列的非 0 非无穷权值个数

6、 $1+n_2+2n_3+\dots+(m-1)n_m$

7、一共 $2n$ 个指针，其中除根结点外均被指向，所以空链域是 $2n-(n-1)=n+1$ ，先序线索化后，仅末尾结点无后继指向，空链域是 1

8、略

9、 $O(\sqrt{x})$

10、 $h-1$ 层的满二叉树+1 层单个结点： 2^{h-1}

二、图的最小生成树算法，如：prim 算法，kruskal 算法。

三、不能，dijkstra 算法不能计算负权边，该算法基于贪婪策略，但所选择的负权边可能导致算法计算出的路径不是全局最短的（需要举例说明）。

四、利用两个栈 S1 和 S2 来模拟一个队列，当需要向队列中插入一个元素时，用栈 S1 来存放入队的元素，即 S1 执行入栈操作。当需要出队时，则对栈 S2 执行出栈操作。由于从栈中取出元素的顺序是原顺序的逆序，所以必须先将 S1 中的所有元素全部出栈并压入到 S2 栈中，再在 S2 中执行出栈操作，即可实现出队操作，而在执行此操作前必须判断 S2 是否为空，否则会导致顺序混乱。当 S1 和 S2 都为空时队列才为空。



```
class MyQueue {
private:
    stack<int> inStack, outStack;

    void in2out() {
        while (!inStack.empty()) {
            outStack.push(inStack.top());
            inStack.pop();
        }
    }

public:
    MyQueue() {}

    void push(int x) {
        inStack.push(x);
    }

    int pop() {
        if (outStack.empty()) {
            in2out();
        }
        int x = outStack.top();
        outStack.pop();
        return x;
    }

    int peek() {
        if (outStack.empty()) {
            in2out();
        }
    }
}
```



```
    }  
    return outStack.top();  
}  
  
bool empty() {  
    return inStack.empty() && outStack.empty();  
}  
};
```

五、

(1) 最简单的用标记数组 flag, 共 target 个元素, flag[i] 标记数字 i 是否存在, 存在则记录下标, 否则为 -1。

(2) 用一个标记数组 flag[target]。

(3) bool findTarget(int num[], int n, int target, int &i, int &j) {

```
    int flag[target] = {-1};  
    for(int k = 0; k < n; ++k) {  
        flag[num[k]] = k;  
        if(flag[target - num[k]] != -1) {  
            i = k;  
            j = flag[target - num[k]];  
            return true;  
        }  
    }
```

```
    }  
    return false;  
}
```

六、推荐使用双指针法, 时间复杂度为 $O(n)$, 但是考虑到考试需要尽可能实现算法得分, 这里以更简单易实现的暴力破解法讲解, 双指针法可以自行上网查阅。暴力破解法使用两层 for 循环枚举每一种可能, 保留最大的存储水量, 时间复杂度为 $O(n^2)$ 。



```
int maxStorage(int height[], int n) {  
    int maxS = 0;  
    for(int i = 0; i < n; ++i) {  
        for(int j = i+1; j < n; ++j) {  
            int h = min(height[i], height[j]);  
            int l = j - i;  
            maxS = max(maxS, h*l);  
        }  
    }  
    return maxS;  
}
```



七、每个盘块大小占用 4KB，可以存储 $4KB/32bit=4KB/4B=1K$ 个盘块。共可以存储的文件大小为 $4KB*(10+1K+1K*1K+1K*1K*1K)=4TB+4GB+4MB+40KB$

八、

(1) s1:就绪态

s2:执行态

s3:阻塞态

(2) t1:进程调度

t2:时间片用完

t3:等待某个事件

t4:所等待事件完成



九、

(1) 共享:b 堆内存和 c 全局变量, 可以通过共享内存机制被多个进程共享。

独占:a 寄存器值是 CPU 内部存储, 无法共享; d 栈内存用于存储局部变量和函数调用, 每个进程都有自己独立的栈。

(2) 共享:b 堆内存, 进程内的所有线程共享同一块堆内存, 线程可以访问和修改堆中的数据; c 全局变量在进程内的所有线程间是共享的, 任何线程对全局变量的修改都会影响其他线程。

独占:a 寄存器值, 寄存器是 CPU 的内部存储单元, 每个线程在执行时有自己独立的寄存器值, 因此不能共享。d 栈内存, 每个线程都有自己的栈内存, 用于存储局部变量、函数调用等, 因此栈内存是线程独立的, 无法共享。

十、

(1) 一页有 $2^{22}B=4MB$, 所有的页共 1K 个, 存储所有页需要 $1K*4B=4KB$ 的连续空间。

(2) n 翻译的物理地址 m 是 A1400000H, 所以 m+1 对应 $A1400000H+2^{22}=A1800000H$ 。

十一、

(1) 略。

(2) 略。

(3) 略。

十二、

semaphore mutex = 1, full1 = 0, full2 = 0, empty = n;

R1 () {

while(true) {

P(empty); // 缓冲区有空位

Produce(data1); // 产生数据

P(mutex);

Put(data1); // 互斥放入数据 1

V(mutex);



```
V(full1);  
}  
}  
  
W1 () {  
while(true) {  
    P(full1); // 缓冲区有数据 1  
    P(mutex);  
    Gut(data1); // 互斥打印数据 1  
    V(mutex);  
    V(empty);  
}  
}
```

```
R2 () {  
while(true) {  
    P(empty); // 缓冲区有空位  
    Produce(data2); // 产生数据  
    P(mutex);  
    Put(data2); // 互斥放入数据 2  
    V(mutex);  
    V(full2);  
}  
}
```

```
W2 () {  
while(true) {  
    P(full2); // 缓冲区有数据 2  
    P(mutex);  
    Gut(data2); // 互斥打印数据 2
```



```
V(mutex);  
V(empty);  
}  
}
```

